

1. Estructura del Árbol de Sintaxis Abstracta

El Árbol de Sintaxis Abstracta (AST) se representa mediante nodos de tipo `NodeAST`. Cada nodo contiene información sobre el símbolo asociado, sus hijos, el tipo de la expresión y el tipo de nodo.

La estructura propuesta es la siguiente:

```
typedef struct NodeAST {
    Symbol *symbol;

    struct NodeAST *left;
    struct NodeAST *right;

    DataType type;
    NodeType nodeType;
} NodeAST;
```

Los tipos de nodo se representan mediante:

```
typedef enum {
    ASSIGNMENT_NODE,
    BINOP_NODE,
    CONSTANT_NODE,
    RETURN_NODE,
    DEFINITION_NODE
} NodeType;
```

Los tipos semánticos de las expresiones se representan mediante:

```
typedef enum {
    TYPE_INT,
    TYPE_BOOL,
    TYPE_VOID
} DataType;
```

Por otro lado, un símbolo contiene únicamente un identificador y un valor:

```
typedef struct Symbol {
    char *id;
    char *value;
} Symbol;
```

El tipo no forma parte de `Symbol`, ya que corresponde a una característica del nodo del AST. De esta manera se evita duplicar la información y mantener dos fuentes de verdad diferentes para el tipo.

2. Representación de símbolos

El campo `symbol` de un nodo no representa exclusivamente variables. También puede utilizarse para representar operaciones o resultados temporales producidos durante la resolución del AST.

Por ejemplo, para la expresión:

$$a + b$$

el nodo correspondiente puede asociarse al símbolo:

```
id = "+"
value = "t1"
```

donde **t1** representa un resultado temporal. Este símbolo no necesariamente debe formar parte de la tabla de símbolos, ya que puede ser información local al AST.

En cambio, para una variable, el símbolo puede corresponder a una entidad persistente de la tabla de símbolos.

3. Reglas de construcción y resolución

Las reglas definidas para los principales tipos de nodos son las siguientes.

3.1. Operación binaria

Para una operación binaria, el operador se almacena en el identificador del símbolo asociado al nodo. Las expresiones izquierda y derecha se resuelven recursivamente y se obtiene un valor temporal para el resultado de la operación.

Dada una expresión:

$$E = E_1 \text{ op } E_2$$

se realizan los siguientes pasos:

1. Se obtiene el operador *op*.
2. Se resuelve la expresión izquierda E_1 .
3. Se resuelve la expresión derecha E_2 .
4. Se aplica la operación correspondiente.
5. Se genera un valor temporal para el resultado.
6. Se determina el tipo de la expresión a partir de los tipos de sus operandos.
7. Se construye un nodo **BINOP_NODE**.

Por ejemplo, para:

$$a + b$$

se puede construir el siguiente nodo:

```
BINOP_NODE
type = TYPE_INT
|
Symbol("+", "t1")
/ \
a b
INT INT
```

El tipo del nodo se obtiene a partir de los tipos de sus hijos:

```
node->type = inferType(  
    node->left->type,  
    node->right->type  
);
```

3.2. Asignación

Para una asignación:

$$x = E$$

el identificador de la variable se obtiene del lado izquierdo, mientras que el valor se obtiene resolviendo la expresión del lado derecho.

El nodo se construye como un **ASSIGNMENT_NODE**, asociando al símbolo el identificador de la variable y el valor producido por la expresión.

Por ejemplo:

```
ASSIGNMENT_NODE  
  type = INT  
  |  
  Symbol("x", "t1")  
  / \  
  x BINOP_NODE  
    type = INT  
    |  
    Symbol("+", "t1")  
    / \  
    a b
```

La resolución conceptual de la asignación es:

```
id = left->symbol->id;  
value = right->symbol->value;  
type = right->type;
```

De esta manera, el tipo de la asignación queda determinado por el tipo de la expresión asignada.

3.3. Retorno

Para una sentencia de retorno:

$$\text{return } E$$

la expresión retornada se almacena como hijo del nodo **RETURN_NODE**.

El tipo del retorno corresponde al tipo de la expresión:

```
RETURN_NODE  
type = expression->type  
|  
expression
```

Por ejemplo, para:

`return a + b`

se obtiene:

```
RETURN_NODE
  type = INT
    |
  BINOP_NODE
    type = INT
      |
    Symbol("+", "t1")
      / \
     a  b
```

El nodo de retorno puede asociarse además a un símbolo con:

```
id = "return"
value = "t1"
```

donde `t1` representa el resultado de la expresión retornada.

3.4. Constantes

Las constantes constituyen hojas del AST, por lo que no poseen hijos.

Por ejemplo, para la constante:

5

se puede construir:

```
CONSTANT_NODE
type = TYPE_INT
  |
Symbol("5", "5")
```

El tipo de una constante se determina directamente a partir de su valor.

4. Regla general de tipado

El tipo se almacena exclusivamente en el campo `type` de `NodeAST`. Para los nodos que representan expresiones compuestas, el tipo puede determinarse a partir de los tipos de sus hijos.

Por lo tanto:

- Las hojas determinan su propio tipo.
- Las operaciones binarias determinan su tipo a partir de los tipos de sus operandos.
- Las asignaciones toman el tipo de la expresión asignada.
- Los retornos toman el tipo de la expresión retornada.

Esta separación permite distinguir entre la información simbólica de una expresión y su información semántica. `Symbol` contiene el identificador y el valor asociado, mientras que `NodeAST` contiene el tipo y la estructura sintáctica de la expresión.

“`latex`

5. Justificación de la refactorización

La refactorización propuesta surge a partir de la forma en que se resolvían los identificadores durante la construcción y posterior procesamiento del AST.

5.1. Implementación anterior

En la implementación anterior, los símbolos se almacenaban en una estructura lineal:

```
Symbol[] symbols;
```

Cada vez que durante la construcción del AST aparecía un identificador, era necesario buscar el símbolo correspondiente dentro de esta estructura:

```
buscar_id(char *id) {  
    for (...) {  
        ...  
    }  
}
```

Por lo tanto, en el peor caso, encontrar un identificador requería recorrer la totalidad de la tabla de símbolos, teniendo un costo de:

$$O(n)$$

donde n representa la cantidad de símbolos almacenados.

Por ejemplo, considerando el siguiente código:

```
int c;  
int d;  
int e;  
int f;  
int a;  
  
a = 5;  
int b = a + a + a;
```

La variable a aparece tres veces en la expresión de inicialización de b . Con la implementación anterior, cada aparición requiere realizar nuevamente una búsqueda sobre la tabla de símbolos.

Si cada búsqueda puede recorrer hasta n elementos, el costo asociado a las tres apariciones es:

$$O(n) + O(n) + O(n) = O(3n)$$

Por lo tanto, aunque el factor constante no modifica la complejidad asintótica, se están realizando búsquedas lineales repetidas para obtener información que ya fue encontrada anteriormente.

5.2. Implementación propuesta

La refactorización consiste en que, una vez encontrado el símbolo correspondiente a un identificador, el nodo del AST almacene directamente una referencia al símbolo original.

En lugar de almacenar únicamente el identificador:

```
ID_NODE
  id = "a"
```

el nodo mantiene un puntero al objeto `Symbol` correspondiente:

```
node->symbol = symbol;
```

De esta forma, los distintos nodos que representan una misma variable pueden mantener una referencia al mismo símbolo:

```
      Symbol "a"
        |
+-----+-----+
|   |   |
ID(a) ID(a) ID(a)
```

Esto permite conservar una única representación del símbolo y mantener consistentes las referencias a la variable a lo largo del AST.

5.3. Costo de acceso

La asignación de un puntero al nodo es una operación de costo constante:

$$O(1)$$

Por lo tanto, una vez resuelto el identificador durante la construcción del AST, acceder posteriormente a su símbolo no requiere volver a recorrer la tabla de símbolos.

Por ejemplo, para obtener el valor asociado a una variable:

```
node->symbol->value
```

el acceso al símbolo se realiza directamente mediante el puntero almacenado en el nodo. En el ejemplo anterior:

```
int c;
int d;
int e;
int f;
int a;

a = 5;
int b = a + a + a;
```

las tres apariciones de `a` mantienen una referencia directa al mismo `Symbol`. Por lo tanto, durante el procesamiento posterior del AST, el acceso a los tres valores tiene un costo de:

$$O(1) + O(1) + O(1) = O(3)$$

5.4. Comparación

La diferencia puede observarse comparando ambas implementaciones. En la implementación anterior, para las tres apariciones de a :

$$O(n) + O(n) + O(n) = O(3n)$$

mientras que en la implementación propuesta:

$$O(1) + O(1) + O(1) = O(3)$$

El costo de encontrar inicialmente cada identificador no desaparece. La búsqueda inicial continúa teniendo un costo de $O(n)$ si la tabla de símbolos se implementa como una estructura lineal. La diferencia consiste en que dicha búsqueda se realiza durante la construcción del AST y el resultado se conserva mediante una referencia directa.

De esta manera, para el ejemplo, podemos representar el costo como:

$$O(n + 3)$$

donde:

- $O(n)$ corresponde a la construcción e inserción de los símbolos y a la resolución inicial necesaria para asociar los identificadores.
- $O(3)$ corresponde a los tres accesos directos a los símbolos mediante los punteros almacenados en los nodos.

Por otro lado, si posteriormente se recorriera el AST y cada aparición del identificador obligara nuevamente a buscar el símbolo en una estructura lineal, el costo asociado sería:

$$O(n + 3n)$$

Por lo tanto, la refactorización no modifica necesariamente la complejidad asintótica total de la construcción del árbol, que continúa siendo lineal bajo las mismas condiciones, sino que evita búsquedas lineales repetidas durante las operaciones posteriores sobre el AST.

5.5. Ventaja principal

La principal ventaja de la refactorización es que cada nodo del AST que representa un identificador mantiene una referencia directa al símbolo que le corresponde.

Esto proporciona dos propiedades importantes:

1. **Acceso constante:** una vez resuelto el identificador, acceder a su información se realiza en $O(1)$ mediante el puntero.
2. **Consistencia de referencias:** distintas apariciones de una misma variable pueden apuntar al mismo objeto `Symbol`, evitando la duplicación de información y permitiendo que todas las referencias representen la misma entidad.

En consecuencia, la tabla de símbolos continúa siendo la estructura responsable de almacenar los símbolos, mientras que el AST conserva referencias directas a aquellos símbolos que necesita durante su procesamiento. ““